

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	K. Pujitha	Reg. No.:	192524149
Section / Batch:	2 nd Year (AI & DS)	Date:	22/07/2026

Code of Conduct

I K. Pujitha (192524149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Pujitha

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic		Focus	Q Nos	Marks
Search Data Structure, Stack		Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure		List Operations	6 – 10	25
Queue Data Structures		Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations		Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

QUESTIONS

Q1
Binary Search
5 marksQ2
Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack
5 marksQ6
Linked List
5 marksQ7
Stack
5 marksQ8
Linked List Data Structures
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

Q1 Binary Search

5 marks

Identify the error in the following snippet.

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;  
    while(low < high) {  
        int mid = (low + high) / 2;  
        if(arr[mid] == key)  
            return mid;  
        else if(arr[mid] < key)  
            low = mid;  
        else  
            high = mid;  
    }  
    return -1;  
}
```

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int binarysearch(int arr[],int n,int key)  
2 {  
3     int low=0,high=n-1;  
4     while (low<=high)  
5     {  
6         int mid=(low+high)/2;  
7         if (arr[mid]==key)  
8             low=mid+1;  
9         else  
10            high=mid-1;  
11    }  
12    return-1;  
13 }
```

Test Input

stdin input (optional)

Output

QUESTIONS

Q1
Binary Search
5 marksQ2
Stack
5 marksQ3
Stack
5 marksQ4
Stack
5 marksQ5
Stack
5 marksQ6
Linked List
5 marksQ7
Stack
5 marksQ8
Linked List Data Structures
5 marksQ9
Linked List
5 marksQ10
Queue
5 marks

Q2 Stack

5 marks

Find the error in the stack deletion operation. Justify it.

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

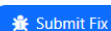
Your Fixed Code

```
1 int pop()  
2 {  
3     if (top==1)  
4     {  
5         printf("stack underflow");  
6         return-1;  
7     }  
8     return stack [top--];  
9 }
```

Test Input

stdin input (optional)

Output



QUESTIONS
Q1
Binary Search
5 marks

Q2
Stack
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Stack
5 marks

Q6
Linked List
5 marks

Q7
Stack
5 marks

Q8
Linked List Data Structures
5 marks

Q9
Linked List
5 marks

Q10
Queue
5 marks

Q3 Stack

5 marks

Check the snippet and identify the logical error.

```
int stack[5];
int top;
void init() {
    top = 0;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 include<stdio.h>
2 int stack[5];
3 int top;
4 void init()
5 {
6     top=-1;
7 }
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

QUESTIONS
Q1
Binary Search
5 marks

Q2
Stack
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Stack
5 marks

Q6
Linked List
5 marks

Q7
Stack
5 marks

Q8
Linked List Data Structures
5 marks

Q9
Linked List
5 marks

Q10
Queue
5 marks

Q4 Stack

5 marks

Identify the errors in the following snippet.

```
if(ch == ' '){
    while(stack[top] != ' '){
        postfix[j++] = pop();
    }
    pop();
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if (ch==' '){
2 {
3     while (top!=-1&&stack[top]!=' '){
4         {
5             postfix[j++]=pop();
6         }
7     pop();
8 }
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

QUESTIONS
Q1
Binary Search
5 marks

Q2
Stack
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Stack
5 marks

Q6
Linked List
5 marks

Q7
Stack
5 marks

Q8
Linked List Data Structures
5 marks

Q9
Linked List
5 marks

Q10
Queue
5 marks

Q5 Stack

5 marks

In balancing Bracket, you identify the error.

```
if((ch == '[' && stack[top] == '[') ||
(ch == ']' && stack[top] == '[') {
    pop();
} else {
    return 0;
}
```

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if(top== -1)
2     return 0;
3 if((ch=='[' && stack[top]=='[') || (ch==']' && stack[top]=='[') || (ch=='[' &&
4 {
5     pop();
6 }
7 else
8 {
9     return 0;
10 }
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

QUESTIONS
Q1
Binary Search
5 marks

Q2
Stack
5 marks

Q3
Stack
5 marks

Q4
Stack
5 marks

Q5
Stack
5 marks

Q6
Linked List
5 marks

Q7
Stack
5 marks

Q8
Linked List Data Structures
5 marks

Q9
Linked List
5 marks

Q10
Queue
5 marks

Q6 Linked List

5 marks

What is the issue?

```
temp = head
while temp.next != NULL:
    print(temp.data)
    temp = temp.next
```

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 temp=head;
2 while (temp!=null)
3 {
4     printf("%d",temp->data);
5     temp=temp->next;
6 }
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

QUESTIONS

- Q1
Binary Search
5 marks
- Q2
Stack
5 marks
- Q3
Stack
5 marks
- Q4
Stack
5 marks
- Q5
Stack
5 marks
- Q6
Linked List
5 marks
- Q7
Stack
5 marks
- Q8
Linked List Data Structures
5 marks
- Q9
Linked List
5 marks
- Q10
Queue
5 marks

Q7 Stack

5 marks

Point out the error.
POP(stack):
if top == 0:
print("Underflow")
else:
return stack[top--]

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 POP(stack)
2 {
3     if (top==1)
4         printf ("underflow");
5     else
6         return stack[top--];
7 }
```

Test Input

stdin input (optional)

Output

Run Save Submit Fix

QUESTIONS

- Q1
Binary Search
5 marks
- Q2
Stack
5 marks
- Q3
Stack
5 marks
- Q4
Stack
5 marks
- Q5
Stack
5 marks
- Q6
Linked List
5 marks
- Q7
Stack
5 marks
- Q8
Linked List Data Structures
5 marks
- Q9
Linked List
5 marks
- Q10
Queue
5 marks

Q8 Linked List Data Structures

5 marks

Identify the errors in the following code
while current != NULL:
current.next = prev
prev = current
current = current.next

✳ **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 while(current!=null)
2 {
3     next=current->next;
4     current->next=prev;
5     prev=current;
6     current=next;
7 }
```

Test Input

stdin input (optional)

Output

Run Save Submit Fix

QUESTIONS

- Q1**
Binary Search
5 marks
- Q2**
Stack
5 marks
- Q3**
Stack
5 marks
- Q4**
Stack
5 marks
- Q5**
Stack
5 marks
- Q6**
Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Linked List Data Structures
5 marks
- Q9**
Linked List
5 marks
- Q10**
Queue
5 marks

Q9 Linked List

5 marks

What is the error in the below code?

```
new.next = head  
head = new
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 new=creatnode(data);  
2 new->next=head;  
3 head=new;
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

Show hidden icons

QUESTIONS

- Q1**
Binary Search
5 marks
- Q2**
Stack
5 marks
- Q3**
Stack
5 marks
- Q4**
Stack
5 marks
- Q5**
Stack
5 marks
- Q6**
Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Linked List Data Structures
5 marks
- Q9**
Linked List
5 marks
- Q10**
Queue
5 marks

Q10 Queue

5 marks

What is missing in the below pseudocode?

```
DEQUEUE(Q):  
if front == -1:  
    print("Underflow")  
else:  
    return Q[front++]
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 DEQUEUE(Q)  
2 {  
3     if (front == -1)  
4         printf("underflow");  
5     else  
6     {  
7         x=Q[front++]  
8         if (front>rear)  
9             front=rear--1;  
10        return x;  
11    }  
12 }
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS
Q1
Stack Notations - Postfix Expression
5 marks

Q2
Circular Queue
5 marks

Q3
Binary Search
5 marks

Q4
Queue
5 marks

Q5
Queue
5 marks

Q6
Doubly Linked List
5 marks

Q7
Stack
5 marks

Q8
Stack
5 marks

Q9
Circular Queue
5 marks

Q1 Stack Notations - Postfix Expression

5 marks

Find the bug in the evaluation of a Postfix expression.

```
while (expression[i] != '\0') {
    if (isdigit(expression[i])) push(expression[i] - '0');
    else {
        int op1 = pop();
        int op2 = pop();
        // perform operation: result = op1 + op2;
        push(result);
    }
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 while (expression[i] != '\0')
2 {
3     if (isdigit(expression[i]))
4         push(expression[i] - '0');
5     else
6     {
7         int op2=pop();
8         int op1=pop();
9         int result=op1+op2;
10        push(result);
11    }
12    i++;
13 }
```

Test Input

stdin input (optional)

Output

▶ Run

📁 Save

🚀 Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS
Q1
Stack Notations - Postfix Expression
5 marks

Q2
Circular Queue
5 marks

Q3
Binary Search
5 marks

Q4
Queue
5 marks

Q5
Queue
5 marks

Q6
Doubly Linked List
5 marks

Q7
Stack
5 marks

Q8
Stack
5 marks

Q9
Circular Queue
5 marks

Q2 Circular Queue

5 marks

Identify the bug in this Circular Queue dequeue logic.

```
int dequeue() {
    if (front == -1) return -1;
    int data = queue[front];
    front = (front + 1) % MAX;
    return data;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int dequeue()
2 {
3     if (front == -1)
4         return -1;
5     int data=queue[front];
6     if (front==rear)
7         front=rear-1;
8     else
9         front=(front+1) % MAX ;
10    return data;
11 }
```

Test Input

stdin input (optional)

Output

▶ Run

📁 Save

🚀 Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q3 Binary Search

5 marks

Identify the bug in this Recursive Binary Search.

```
int binarySearch(int arr[], int l, int r, int x) {  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == x) return mid;  
    if (arr[mid] > x) return binarySearch(arr, l, mid, x);  
    return binarySearch(arr, mid, r, x);  
}
```

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int binarysearch(int arr[],int l,  
2     int r,int x)  
3 {  
4     if(l>r) return -1;  
5     int mid = l + (r-1)/2;  
6     if(arr[mid]==x) return mid;  
7     if(arr[mid]>x)  
8         return binarysearch(arr,l,mid-1,x);  
9     return binarysearch (arr,mid+1,r,x);  
10 }
```

Test Input

stdin input (optional)

Output

▶ Run

💾 Save

✳ Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1

Stack Notations - Postfix
Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q4 Queue

5 marks

Find the error in this Deque insertFront operation.

```
void insertFront(int data) {  
    if (front == 0) front = MAX - 1;  
    else front = front - 1;  
    deque[front] = data;  
}
```

✳ Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 void insertfront(int data)  
2 {  
3     if (front==1)  
4         front=rear+1;  
5     else if (front==0)  
6         front = MAX -1;  
7     else  
8         front--;  
9     deque[front]=data;  
10 }
```

Test Input

stdin input (optional)

Output

▶ Run

💾 Save

✳ Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

- Q1**
Stack Notations - Postfix Expression
5 marks
- Q2**
Circular Queue
5 marks
- Q3**
Binary Search
5 marks
- Q4**
Queue
5 marks
- Q5**
Queue
5 marks
- Q6**
Doubly Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Stack
5 marks
- Q9**
Circular Queue
5 marks

Q5 Queue

5 marks

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {  
    if (rear == MAX) return;  
    queue[rear++] = data;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 void enqueue(int data)  
2 {  
3     if (rear == MAX - 1) return ;  
4     queue[++rear]=data;  
5 }
```

Test Input

stdin input (optional)

Output

[▶ Run](#) [☁ Save](#) [🌟 Submit Fix](#)

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

[← Back](#)

QUESTIONS

- Q1**
Stack Notations - Postfix Expression
5 marks
- Q2**
Circular Queue
5 marks
- Q3**
Binary Search
5 marks
- Q4**
Queue
5 marks
- Q5**
Queue
5 marks
- Q6**
Doubly Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Stack
5 marks
- Q9**
Circular Queue
5 marks

Q6 Doubly Linked List

5 marks

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {  
    if (del->prev != NULL)  
        del->prev->next = del->next;  
    if (del->next != NULL)  
        del->next->prev = del->prev;  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 void deletenode(node*del)  
2 {  
3     if (del->prev !=NULL)  
4         del->prev->next=del->next;  
5     if (del->next !=NULL)  
6         del->next->prev=del->prev;  
7 }
```

Test Input

stdin input (optional)

Output

[▶ Run](#) [☁ Save](#) [🌟 Submit Fix](#)

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

- Q1**
Stack Notations - Postfix Expression
5 marks
- Q2**
Circular Queue
5 marks
- Q3**
Binary Search
5 marks
- Q4**
Queue
5 marks
- Q5**
Queue
5 marks
- Q6**
Doubly Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Stack
5 marks
- Q9**
Circular Queue
5 marks

Q7 Stack

5 marks

Identify the issue in the below pseudocode.
POP():
if top == 0 → Underflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 POP() :
2 if top == -1 -> underflow
3 else
4 return stack[top--]
```

Test Input

POP(

Output

Run Save Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

- Q1**
Stack Notations - Postfix Expression
5 marks
- Q2**
Circular Queue
5 marks
- Q3**
Binary Search
5 marks
- Q4**
Queue
5 marks
- Q5**
Queue
5 marks
- Q6**
Doubly Linked List
5 marks
- Q7**
Stack
5 marks
- Q8**
Stack

Q8 Stack

5 marks

Point out the error.
if top == MAX:
Overflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if top == max-1:
2 Overflow
```

Test Input

stdin input (optional)

Output

Run Save Submit Fix

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q9** Circular Queue

5 marks

Identify the issue in the following snippet for Circular Queue in Traffic Signal.

```
rear = (rear + 1) % MAX;  
queue[rear] = vehicle;  
if(rear == front){  
    printf("Queue Full");  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 if((rear+1) % MAX ==front)  
2 {  
3     print("Queue Full");  
4 }  
5 else  
6 {  
7     rear = (rear+1) % MAX;  
8     queue [rear]=vehicle;  
9 }
```

Test Input

stdin input (optional)

Output

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

← Back

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks**Q2**
Circular Queue
5 marks**Q3**
Binary Search
5 marks**Q4**
Queue
5 marks**Q5**
Queue
5 marks**Q6**
Doubly Linked List
5 marks**Q7**
Stack
5 marks**Q10** Stack

5 marks

Point out the error?

```
PEEK(stack):  
return stack[top+1]
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 int PEEK (int stack[],int top)  
2 {  
3     if (top== -1)  
4     {  
5         printf ("stack underflow\n");  
6         return -1;  
7     }  
8     return stack[top];  
9 }  
10
```

Test Input

stdin input (optional)

Output

Run

Save

Submit Fix